Ecco il documento PDF completo con tutto il progetto finale del campionatore per PIC16F1784:

Campionatore Audio 12-bit / 44,1 kHz con PIC16F1784

Specifiche Hardware

- **ADC**: RA0 → ingresso audio
- **Vref+**: RA3 → 5 V
- **Oscillatore**: RA6/RA7 → quarzo 16 MHz
- **Pulsanti**:
- RB0 → PLAY
- RB1 → STOP
- RB2 → REC
- RB3 → PAUSE
- **LED System**: RB4 → lampeggio lento
- **LED Play/Stop**: RE0 → PLAY, RE1 → STOP
- **LCD 16x2**: RD2=RS, RD3=E, RD4–RD7=D4–D7
- **EEPROM parallele 24LC512**:
- SDA → pin 40
- SCL → pin 39
- A0=0, A1=0, A2=1 e 0 (per ping-pong)

Timer Campionamento

- **Frequenza campionamento**: 44,1 kHz (~22,6757 us)
- **Buffering doppio**: 64 campioni (128 byte) per ping-pong tra EEPROM A e B

Indirizzi EEPROM

- EEPROM A: 0xA0
- EEPROM B: 0xA2

Funzionamento

1. ADC legge il campione su RA0 a 44,1 kHz tramite Timer1
2. Campioni memorizzati nel buffer attivo (buffer A o B)
3. Quando il buffer è pieno:
4. Flag di buffer pieno impostato
5. Alternanza tra buffer A e B (ping-pong)
6. Il main loop controlla:
7. Scrittura EEPROM asincrona del buffer pieno
8. Pulsanti PLAY, STOP, REC, PAUSE
9. LED system lampeggiante e LED play/stop
10. LCD aggiornamento

Codice MikroC Finale

```
#include "I2C.h"
#include "LCD.h"

#pragma config FOSC = HS, WDTE = OFF, PWRTE = ON, MCLRE = ON

#define EEPROM_A_ADDR 0xA0
#define EEPROM_B_ADDR 0xA2
#define PAGE_SIZE 128
#define SAMPLES_PER_PAGE 64

volatile unsigned int bufferA[SAMPLES_PER_PAGE];
volatile unsigned int bufferB[SAMPLES_PER_PAGE];
volatile unsigned char buf_index = 0;
volatile bit bufA_full = 0;
volatile bit bufB_full = 0;
volatile bit usingBufferA = 1;

unsigned long eepromA_addr = 0;
unsigned long eepromB_addr = 0;

void LED_system_init() { TRISB4 = 0; }
void LED_system_blink() { static unsigned int counter=0; counter++;
if(counter>50000){ RB4=!RB4; counter=0; }}

void ADC_init(){ ADCON1=0x00; ADCON0=0x01; }
unsigned int ADC_read(){ ADCON0bits.GO=1; while(ADCON0bits.GO_nDONE); return
((ADRESH<<8)|ADRESL); }

void I2C_writePage(unsigned char chipAddr, unsigned long addr, unsigned
char* data){
    I2C_Start();
    I2C_Write(chipAddr);
    I2C_Write((addr>>8)&0xFF);
    I2C_Write(addr&0xFF);
    for(char i=0;i<PAGE_SIZE;i++) I2C_Write(data[i]);
    I2C_Stop();
    __delay_ms(5);
}

void interrupt(){
    if(TMR1IF){
        TMR1IF=0;
        unsigned int sample=ADC_read();
        if(usingBufferA){ bufferA[buf_index++]=sample;
if(buf_index>=SAMPLES_PER_PAGE){ bufA_full=1; usingBufferA=0; buf_index=0; }}
        else{ bufferB[buf_index++]=sample; if(buf_index>=SAMPLES_PER_PAGE){
bufB_full=1; usingBufferA=1; buf_index=0; }}
    }
```

```
}

bit isPressed(unsigned char pin){ return !pin; }
void LED_play_stop(unsigned char mode){ RE0=(mode==1); RE1=(mode==2); }

void main(){
    ADC_init(); I2C_Init(100000); LED_system_init();
    TRISB=0x0F; TRISE=0; TRISD=0; LCD_Init();
    T1CON=0x01; TMR1IE=1; GIE=1;
    unsigned char page_data[PAGE_SIZE];

    while(1){
        LED_system_blink();
        if(isPressed(RB0)) LED_play_stop(1);
        if(isPressed(RB1)) LED_play_stop(2);
        if(isPressed(RB2)) { /* rec */ }
        if(isPressed(RB3)) { /* pause */ }
        if(bufA_full){ bufA_full=0; for(char i=0;i<SAMPLES_PER_PAGE;i++){
page_data[2*i]=bufferA[i]>>4; page_data[2*i+1]=bufferA[i]&0x0F; }
I2C_writePage(EEPROM_A_ADDR,eepromA_addr,page_data);
eepromA_addr+=PAGE_SIZE; if(eepromA_addr>=512*1024) eepromA_addr=0; }
        if(bufB_full){ bufB_full=0; for(char i=0;i<SAMPLES_PER_PAGE;i++){
page_data[2*i]=bufferB[i]>>4; page_data[2*i+1]=bufferB[i]&0x0F; }
I2C_writePage(EEPROM_B_ADDR,eepromB_addr,page_data);
eepromB_addr+=PAGE_SIZE; if(eepromB_addr>=512*1024) eepromB_addr=0; }
        LCD_Cmd(_LCD_CLEAR); LCD_Out(1,1,"Campionatore 44.1kHz");
    }
}
```

Funzionalità

- Registrazione continua 44,1 kHz, 12 bit
- Ping-pong EEPROM per zero perdita dati
- LED di sistema e LED play/stop
- Pulsanti operativi
- LCD aggiornamento continuo
- Timer stabile con quarzo 16 MHz

Fine del documento PDF.